How to Optimize Your Single Agent or Agent Node

Background

In the Smart platform, an Agent (or an Agent Node in a Multi-Agent setup) ultimately delivers results through the combination of four components:

- **`Prompt`** (role & task definition)
- **`Tools`** (function calling)
- **`KB`** (RAG retrieval)
- **`LLM`** (large model capability)

Among these, **`Tools`** are typically fixed code — they either execute successfully or fail; they don’t adapt by themselves. **`KBs`** can vary (e.g., chunking strategy, retrieval method), but such optimization is discussed in How to prepare knowledge for your AI Agent?

Also, we received lots of the question - "agent sends the sensitive info, what should i do?", "agent always says yes with the question it cannot help, what should i do?", " i cannot get the expected answer even tho all tools are ready.", which are more about Prompt and LLM refine. These two factors will largely determine whether your Agent can consistently execute tasks as intended.

When these components come together in the Agent, the key determinant of whether the Agent can follow our intended steps and deliver the expected outcome is the **`Prompt`** (clarity of steps, input/output formats, validation rules) working with the **`LLM`** (reasoning ability, understanding, and adherence).

Quickly Locating the Problem

You can identify the root cause using message **`debug info (tree view + Tool/KB input/output)`**:

1. **`Tool/KB input is correct but output is wrong`**
   → Check **`Tool/KB`** first.
   - Tool: Test tool if runs properly in tool test window.
   - KB: Test KB retrieval for target keywords.
     - If retrieval works, adjust KB advanced settings and parameters to let retrieved chunks could be sent to summarization LLM.
     - If not, restructure KB segmentation or request feature support from the Smart team.
2. **`Tool/KB output is correct, but the Agent uses/combines it incorrectly`**
   → Likely a **`Prompt issue`** (format or instructions not clearly defined). This often happens when the Agent needs to call Tools or KBs multiple times in one task. Even if each single call returns the correct result, problems may occur if:
   - The output from earlier Tool/KB calls is **`wrongly modified by agent`**.
   - The output from earlier Tool/KB calls is **`not correctly recorded into the Agent’s context`**.
   - Retrieved or generated content is **`not passed to later Tool/KB calls`** as required.
3. **`Required Tool/KB is not triggered`** (triggered in wrong order or missing calls)
   → may have several reasons
   - Most cases related to this because users **`didn't explicit @KB/@Tool`** references in the Prompt. Or didn't assign a clear name and description for KB/Tool (name and desc are the key information for LLM to decide when to call)
   - Name and description of Tool/KB are not clear enough to Agent.
   - **`Unclear task or step decomposition`** in prompt
   - Poor instruction following of LLM
4. **`Same Prompt behaves differently on different LLMs, One is much better than another one.`**
   → The bottleneck may be **`LLM capability`**, so switching models might be necessary.

   → Could also consider breaking down the single agent into a multi-agent setup, where a tough task is decomposed into smaller subtasks and assigned to specialized nodes, allowing each node to focus on one task.

Prompt or LLM: Which to Tweak First?

Beyond the above discussion, some problems originate from the Prompt, while others are due to the LLM. So how should you decide which to tweak first?

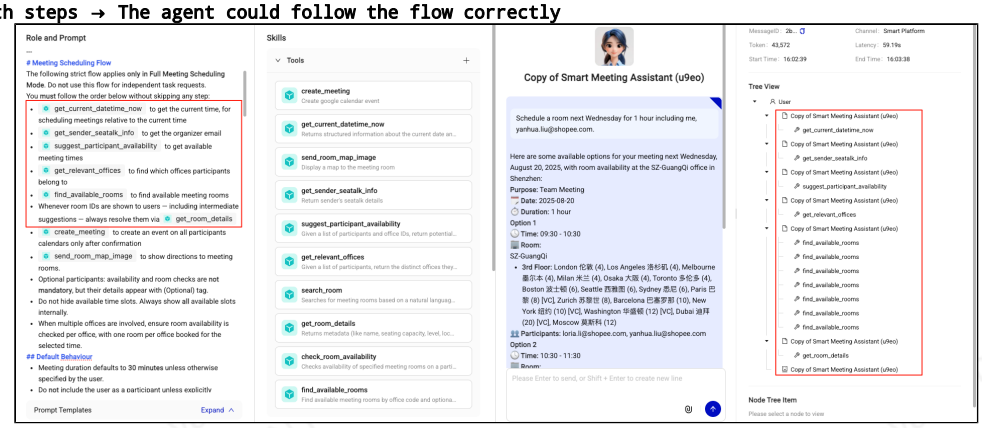
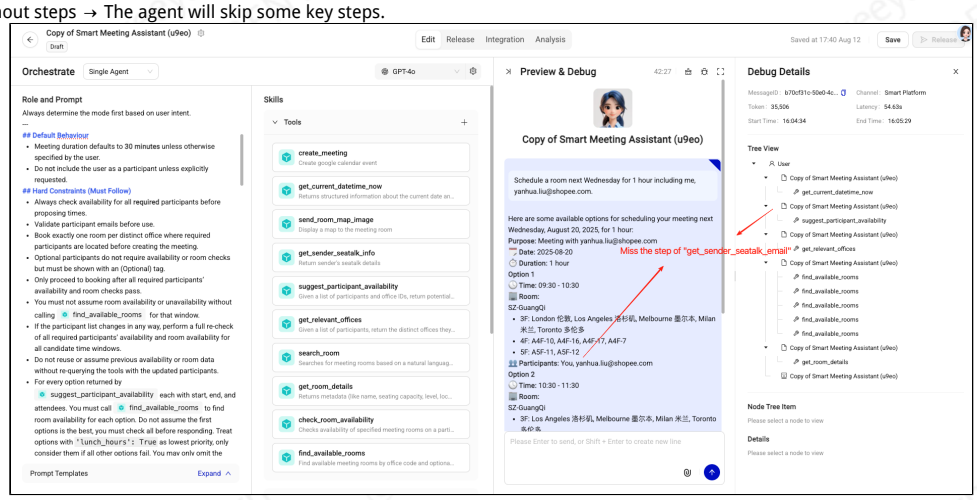
1. **`Suggest to Optimize Prompt first`**
    - Lower cost, faster iteration.
    - Most cases come from unclear instructions, missing output format, unclear steps, or language mismatch with the LLM.
    - After changes, batch test to verify.
2. **`If issues persist after Prompt optimization`**, consider changing/upgrading the LLM to see if the question is still resist
    - if its gone, consider to upgrade ur LLM for agent
    - if not, continue refining ur prompt

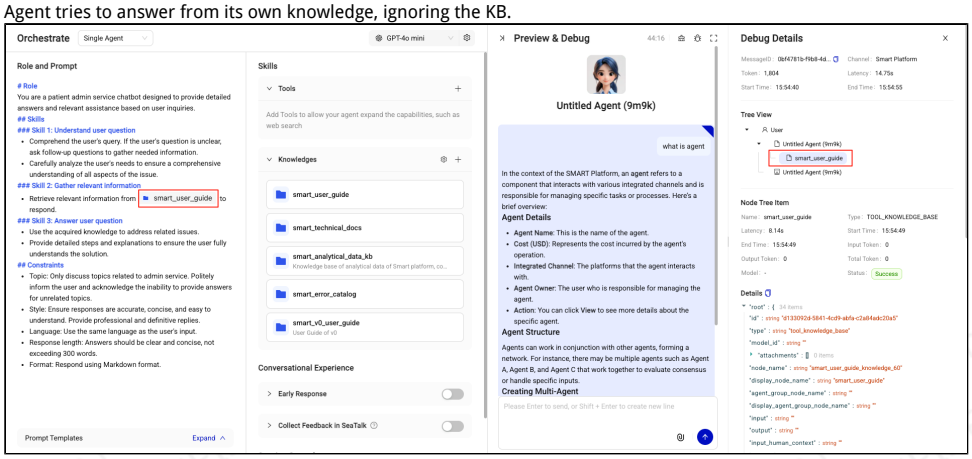
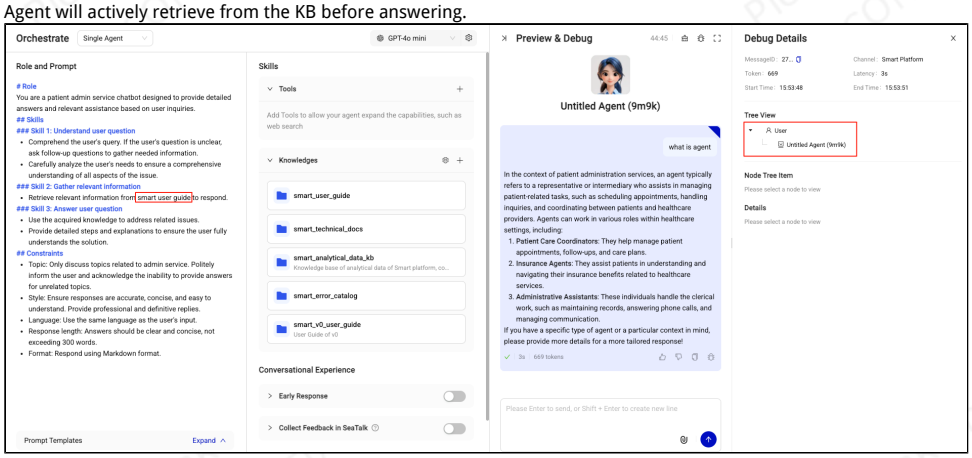
`Tip:` While tweaking Prompts, you can run small-scale LLM comparisons for reference, but don't switch models as the first step.

How to Optimize a Prompt

Here we list down some useful skills we found from real cases, for more technical skills -> Agent Prompt

	DO	DO NOT	Why?	Example
Language Alignment	Write prompts in **English** when targeting OpenAI/Claude/Gemini	Force prompts in Chinese or other language for instruction-following	English has stronger instruction adherence in these LLMs	• In scenarios where the agent needs to support multi-language responses, its very unstable when telling the LLM in Chinese to res English. • b. Follow these rules: i. If the user asked in English, respond in English. ii. If the user asked in Chinese, respond in Chinese. iii. If the user asked in Vietnamese, respond in Vietnamese. iv. If the user asked in Portuguese, respond in Portuguese.
Few-shot Examples	Provide 2–3 **few-shot examples**	Provide zero examples and expect perfect generalization	Few-shots help LLM ground its behavior	• ### Examples: • Project Balance not enough Error ◦ Input (raw) [error_smart_platform] Error from executing the graph. Error=error:failed to invoke graph. Error: Agent android_mr_analyzer_agent_2817 failed to generate response. Error=Error code: 402 - {'message': 'Project balance not enough: team: MP Dev & QA balance not enough', 'retcode': 40200} ◦ Output (processed) Diagnostic: The ask_human_test_agent failed to generate a response, possibly because: - Project MP Dev & QA balance not enough Suggestion: Please ask team's admin to apply for additional quota.

Clarity in Prompt Content	Add **step-by-step instructions** for flow-based tasks **Note**: it's really useful for non-reasoning model. However for reasoning model, as they have excellent capacity of thinking, they may return excellent result with less instruction.	Give vague task description without steps	Clear structure reduces tool/KB call mistakes	• **With steps → The agent could follow the flow correctly**  • Without steps → The agent will skip some key steps. 

	Explicitly use **@KB/@Tool** notation	Add tool/KB in Skills column only, or just describe tool/KB usage in natural language	Ensures LLM triggers the correct resource	• With @KB/@Tool → The Agent clearly understands which resource to invoke, reducing ambiguity.
				  • Agent tries to answer from its own knowledge, ignoring the KB.
				  
				• Without @KB/@Tool (pure text) → The Agent may rely on its own understanding and skip calling the KB/Tool, even if it is configur
				  • Agent will actively retrieve from the KB before answering.
				  

	Add **constraints** like "DO NOT modify Tool/KB outputs"	Leave Tool/KB post-processing unspecified	Avoids agent hallucination or overwriting KB outputs	• Example prompt: Use @get_current_timestamp to get current timestamp and convert the timestamp to local Singapore time.
				• Example prompt:
				  • Add ##Constraint or after calling KB/Tool :
				  • "DO NOT overwrite or modify Tool/KB outputs. Always record them into the working context and directly use them in l
				  • "DO NOT assume something without any knowledge base basis. Because this is internal Sea's platform."

	Add **self-check rules** with max attempts	Add recursive "check again until perfect"	Prevents infinite loops	
	Use LLM to **refine prompts** iteratively	Expect the first draft prompt to work perfectly	Prompt optimization is iterative	• Create a test Agent with the same LLM • Different LLM has different prompt adaptions • Describe requirements and send prompt template to the LLM to generate an initial Prompt. • Run the Prompt, review results, and feed prompt & results back to agent for analysis. • Iterate until results are satisfactory.

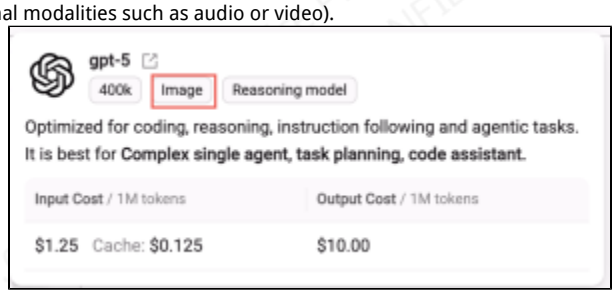
How to Choose an LLM?

Understanding LLM Naming and Types by Provider

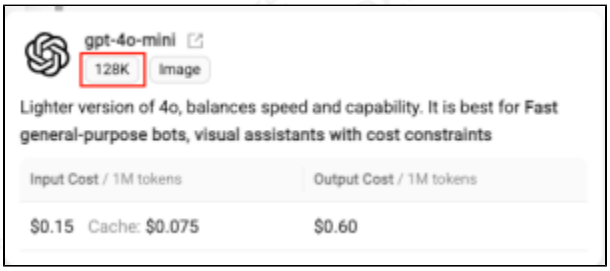
By Core Capacity:

- **General Models** – Cover a wide range of tasks such as writing, Q&A, translation, and content generation. These are commonly used for most business agents.
- **Reasoning Models** – Specifically optimized for multi-step reasoning, and complex decision-making. They will think before return the result, usually come with higher computational costs and **longer latency.**

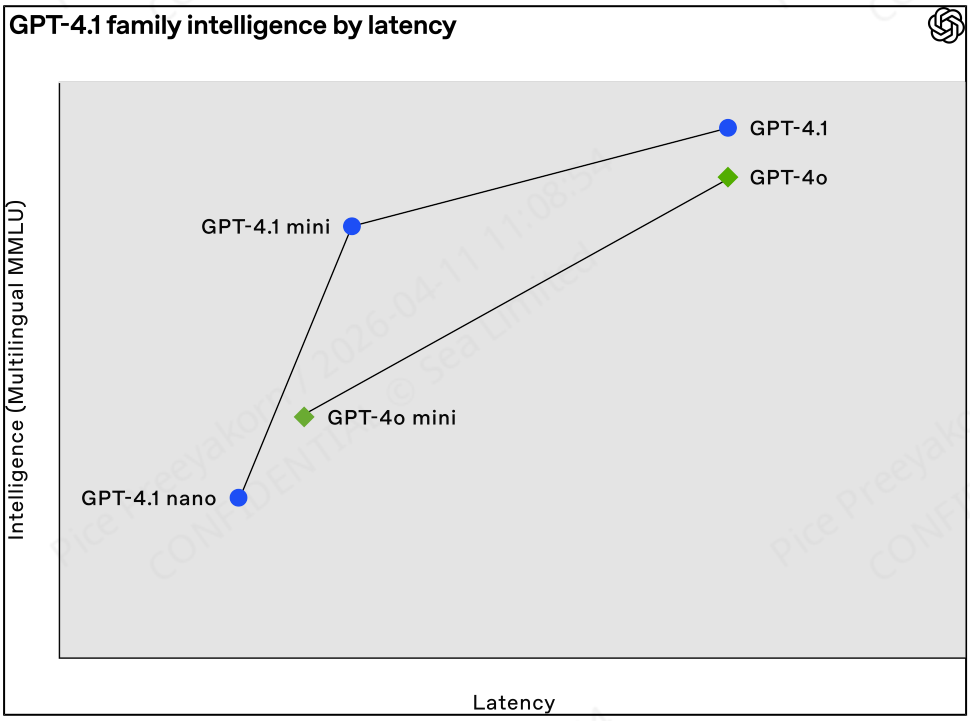
By Input Modality:

- **Text-only Models** – Accept only text-based inputs.
- **Multimodal Models** – Accept text plus other modalities such as images (image input support is common; some models also support additional modalities such as audio or video).
  - 

By Context Memory (Context Window):

- Measured in tokens. For example, a context window of **128K** means the model can process and "remember" up to 128,000 tokens within a single conversation.
  - **English** – Roughly 1 word ≈ 1 token.
  - **Chinese** – Roughly 1–2 characters ≈ 1 token.
  - Larger context windows enable long-document understanding, but may come with higher cost and memory requirements.
  - 

Provider	Focus	Naming Rules / Levels	Level Meaning
ALL		**Version Numbers**- 4, 4.1, 4o, 5 for GPT 3, 3.5, 3.7, 4 for Claude	Higher indicates a newer version with generational technology upgrades, such as improved scores in coding and math benchma Note: However, **a newer model is not necessarily better for every business case** – it might focus more on expense of writing skills, or its expanded knowledge could result in longer latency.

OpenAI	Balances versatility with commercial maturity; well-developed ecosystem; rich API and plugin support. **We recommend to start with GPT series first.**	**Series Types**- • GPT series (e.g., GPT-4, GPT-4.1, GPT-4o, GPT-5) • O series (o3, o4)	• **GPT series**: General-purpose large models suitable for dialogue, writing, analysis, and multi-scenario applications • **O series**: Models optimized for reasoning and complex logic (o3 focuses on medium-level reasoning; o4 targets higher-l
		Tiers- Standard / Mini / Nano	• **Standard**: Full capabilities and highest intelligence. • **Mini**: Faster speed, lower cost, suitable for lightweight scenarios. • **Nano:** Extremely lightweight with lower intelligence, best for simple tasks and high-volume usage. 
Claude (Anthropic)	Best in **programming** and **high-quality writing scenarios**	**Tiers** - Sonnet / Haiku	• **Sonnet:** Most popular LLM in the Claude family. Excels at high-quality content generation and programming tasks — inte independently write and debug complex code. • **Haiku:** Fastest and most cost-efficient, ideal for high-throughput interactions.

Gemini (Google DeepMind)	Industry leader in **long-text processing** (ultra-long context) and multimodal understanding, especially for knowledge bases and large document analysis.	**Tiers** - Pro / Flash	• **Pro**: Full capabilities, suitable for high-precision tasks. • **Flash**: Faster and cheaper, ideal for streaming conversations or batch processing.
Qwen (Alibaba)	It is **free** and locally deployed, with strong **security** and privacy advantages.	**Series Types**- Instruct / VL Instruct / QwQ	• **Instruct**: heavily fine-tuned to follow natural language instructions. They are especially suited for dialogue, creative gen JSON) • **VL Instruct:** An instruction-tuned model with multimodal capabilities. with the capacity pf understanding image and v • **QwQ:** Optimized for reasoning, mathematical and coding tasks.
		Model Size- 7B /32B/72B	Represents the model's parameter size (intelligence level). Larger models are generally smarter but have longer latency, while smaller models respond faster.

Common Pitfalls to Avoid

1. **Assuming newest = best**
    - A newer model might have better instruction-following but different writing style or behavior, which could break existing workflows.
2. **Overusing high-reasoning models in high-concurrency scenarios**
    - o-series and other reasoning models can introduce noticeable latency under load, hurting user experience.
3. **Ignoring "start small" strategy**
    - Jumping directly to the most expensive model without testing may lead to unnecessary costs.
    - **Best practice:** Start with **GPT-4o mini** as the baseline, then switch to stronger LLMs if:
        - The same prompt works significantly better on an upgraded model.
        - Business impact justifies cost.
        - Security or compliance requires local deployment.
4. **Blindly trusting "free = good enough"**
    - Free/open-source (e.g., Qwen) is great for privacy & offline deployment but may underperform in complex reasoning or niche professional domains.